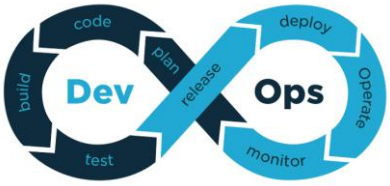
FATIMA JINNAH WOMEN UNIVERSITY
RAWALPINDI

Cloud DevOps



Lab 02

Hands-On Git and Version Control

Submitted by: Rabbia Noor
Registration No.: 2023-BCS-070
Course Code: CS-649
GitHub Username: Rabbia-Noor
Section: A

Table of Contents

Getting Started: Install Git 3
Task 1: Create Private GitHub Repository 3
Task 2: Connect Repository via SSH 3
Task 3: Configure Git Username and Email 4
Task 4: Explore the .git Folder 5
Task 5: Local Repository Management 5
Task 6: File Status & Staging 6
Task 7: Branch Creation Using GitHub GUI 6
Task 8: Branch Creation and Push Using Git Bash 7
Task 9: Branching & Merging 7
Task 10: Pull Request and Branch Review (GitHub GUI) 8
Task 11: Detailed Branch Strategy (Develop/Staging) 9
Task 12: Code Review Workflow 11
Task 13: Branch Cleanup Best Practices 14

Getting Started: Install Git

Objective: Install Git on the PC so that version control commands can be used throughout the lab.

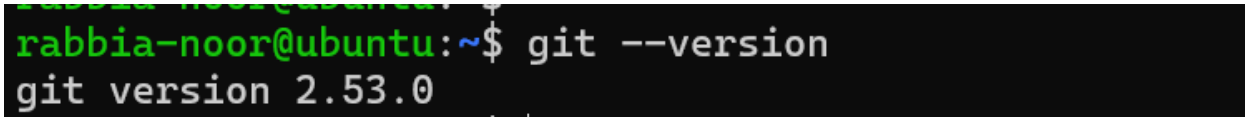


Figure 1: *Git was downloaded and installed on the PC. The Git Bash, Git CMD and Git GUI shortcuts are visible, which confirms the installation completed successfully. These tools are used for all version control work in this lab.*

Task 1: Create Private GitHub Repository

Objective: Create a private repository on GitHub to practice Git commands.

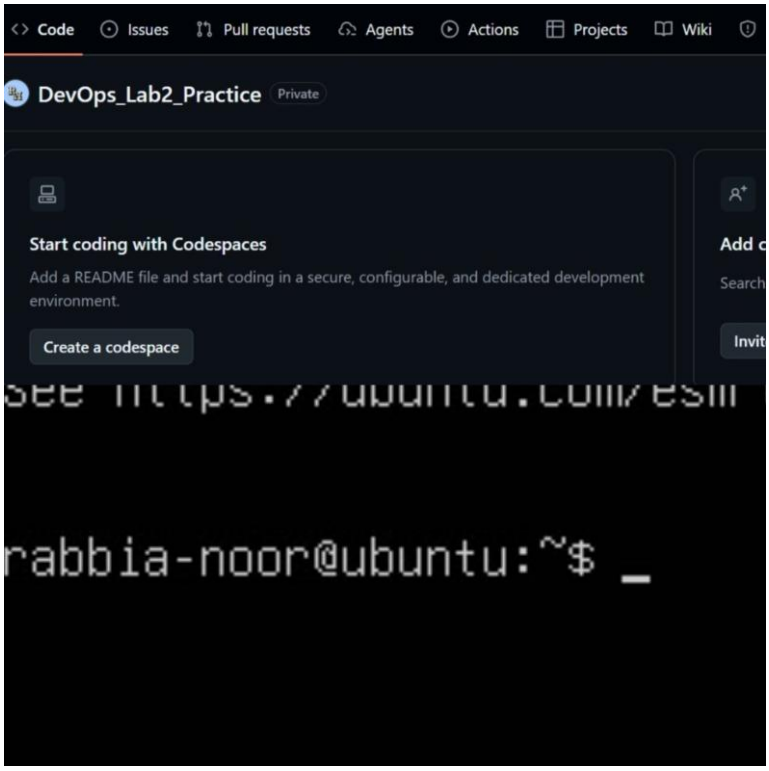


Figure 2: *A new private repository named DevOps_Lab2_Practice was created on GitHub, and the Private label is visible. The terminal prompt rabbia-noor@ubuntu is shown beside it as the required identity proof.*

Task 2: Connect Repository via SSH

Objective: Connect the local machine to GitHub securely using an SSH key instead of a password.



Figure 3: *A new ED25519 SSH key pair was generated with ssh-keygen, and the public key was displayed using cat. Only the public key is shown, so the private key stays secret. This key is used to log in to GitHub securely without a password.*

Step 2: Add the SSH public key to GitHub (Settings > SSH and GPG keys > New SSH key).

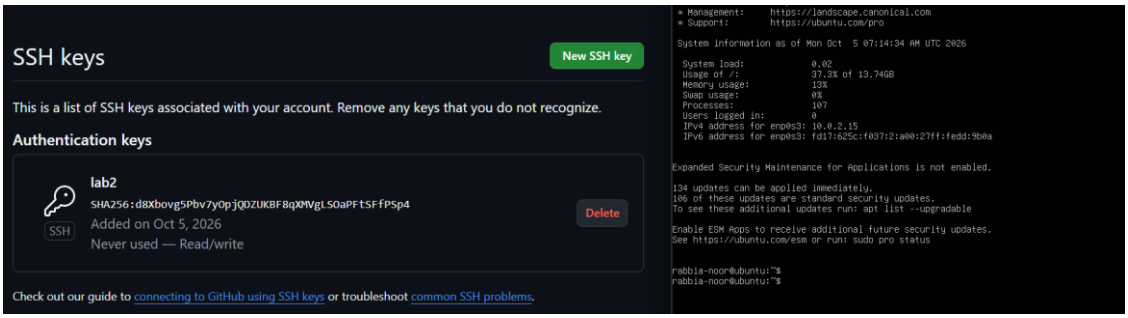


Figure 4: *The public SSH key was added to GitHub under Settings > SSH and GPG keys. This links the key to my GitHub account, so GitHub can recognise my computer and allow secure pushes and pulls over SSH.*

Step 3: Clone the DevOps_Lab2_Practice repository using SSH.



Figure 5: *The repository was cloned to the local machine using its SSH URL (git@github.com). The message about an empty repository is expected for a new repo. A successful clone proves the SSH connection to GitHub works.*

Task 3: Configure Git Username and Email

Objective: Set up the Git identity so that all commits are linked to the correct user.

Step 1: Configure the global Git username and email.

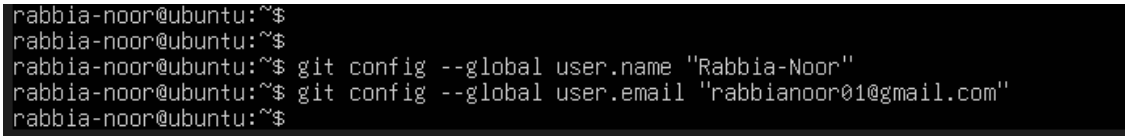


Figure 6: *The Git username and email were set with git config --global. Git attaches this identity to every commit, so all my work is linked to me. It only needs to be configured once on the machine.*

Step 2: Verify the configuration.



Figure 7: *The git config --list command displays all saved Git settings. The output shows user.name=Rabbia-Noor and my email, which verifies that the identity from the previous step was stored correctly.*

Task 4: Explore the .git Folder

Objective: Understand what the hidden .git folder contains.

Step 1: Navigate into the cloned repository folder and show hidden files to locate the .git directory.

Step 2: Explore the contents of the .git folder.

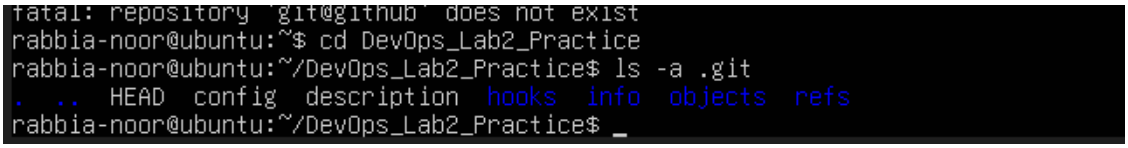


Figure 8: *Inside the cloned repository, ls -a .git lists the hidden .git folder contents such as HEAD, config, hooks, objects and refs. This folder stores the full history and settings of the repository and must not be edited manually.*

Task 5: Local Repository Management

Objective: Learn how a repository is created, committed and connected to a remote.

Step 1: Delete the existing .git folder and run git status. The command gives an error because the directory is no longer a Git repository.

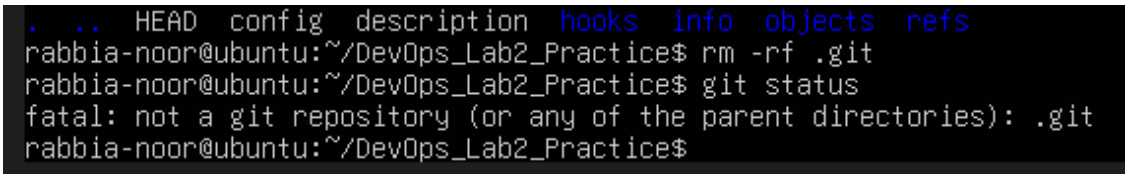


Figure 9: *The .git folder was deleted with rm -rf .git, so the project is no longer a repository. Running git status now fails with "not a git repository". This shows that the .git folder is what makes a directory a Git repository.*

Step 2: Re-initialize the local Git repository.

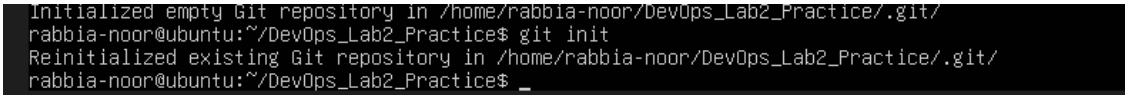


Figure 10: *git init was run to create a new, empty local repository in the project folder. This recreates the .git folder that was deleted earlier, so Git can start tracking the files in this directory again.*

Step 3: Add a file named README.md and commit it.



Figure 11: *A README.md file was created, added to the staging area with git add, and saved with git commit -m "Initial commit". This is the first snapshot in the repository history and shows the basic add-and-commit workflow.*

Step 4: Connect the local repository to GitHub and push.

```
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git remote set-url origin git@github.com:Rabbia-Noor/DevOps_Lab2_Practice.git
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git branch -M main
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 239 bytes | 10.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:Rabbia-Noor/DevOps_Lab2_Practice.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 12: *The local repository was connected to GitHub with git remote add origin, the branch was renamed to main, and the first commit was pushed with git push -u origin main. The project is now stored on GitHub.*

Task 6: File Status & Staging

Objective: Understand the untracked, staged and committed states of a file.

Step 1: Create a new file notes.txt, write a note in it and check the status.

```
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ echo "Adding some notes" >> notes.txt
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        notes.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Figure 13: *notes.txt was created with some text, and git status lists it under untracked files. Git sees the new file but is not tracking it yet. It must be staged with git add before it can be committed.*

Step 2: Stage and commit the file.

```
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git add notes.txt
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git commit -m " Add notes.txt"
[main 4636be1]  Add notes.txt
 1 file changed, 1 insertion(+)
 create mode 100644 notes.txt
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 14: *notes.txt was staged with git add and committed with the message "Add notes.txt". The commit output confirms one file was created. This completes the cycle of untracked, staged and committed.*

Task 7: Branch Creation Using GitHub GUI

Objective: Create a branch from the GitHub web interface and sync it locally.

Step 1: On GitHub, create a branch named bugfix/user-auth-error.



Figure 15: *The branch bugfix/user-auth-error was created from the GitHub web interface. The branch list shows it next to main. Creating branches on GitHub is useful for quickly starting an isolated fix without using the terminal.*

Step 2: Pull the branch to the local repository to sync it.

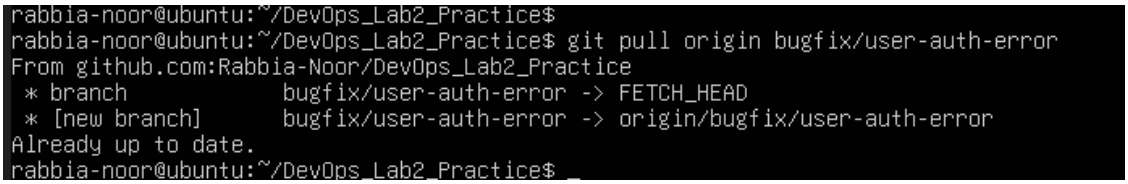


Figure 16: *The new GitHub branch was pulled into the local repository with git pull origin bugfix/user-auth-error. The output shows the remote branch was fetched, which keeps the local and remote copies of the repository in sync.*

Task 8: Branch Creation and Push Using Git Bash

Objective: Create a branch locally and push it to the remote repository.

Step 1: Create a branch named feature/db-connection and push it to the remote repository.

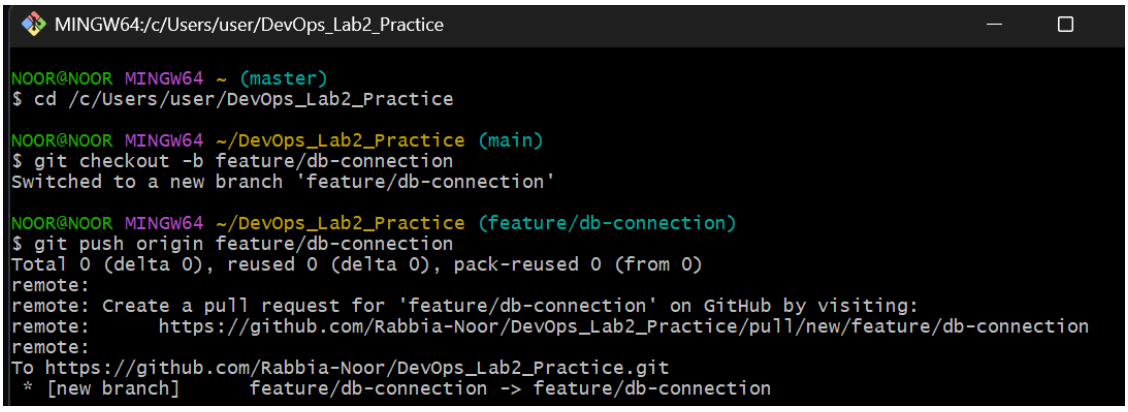


Figure 17: *The branch feature/db-connection was created locally with git checkout -b and pushed to GitHub with git push origin. The output confirms a new remote branch was created, which makes the work available for a Pull Request.*

Task 9: Branching & Merging

Objective: Practice creating a branch, committing on it and merging it back into main.

Step 1: Create and switch to a branch named feature-1.

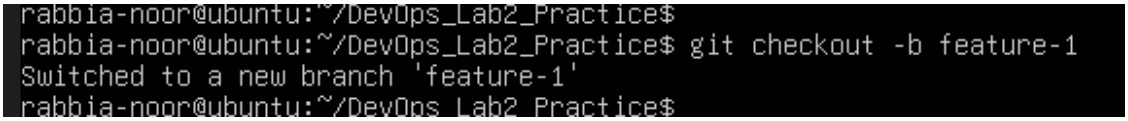


Figure 18: *The command git checkout -b feature-1 created a new branch and switched to it in one step. Working on a separate branch keeps new development isolated from the stable main branch until it is ready to merge.*

Step 2: Modify main.py (add a function) and commit the change.

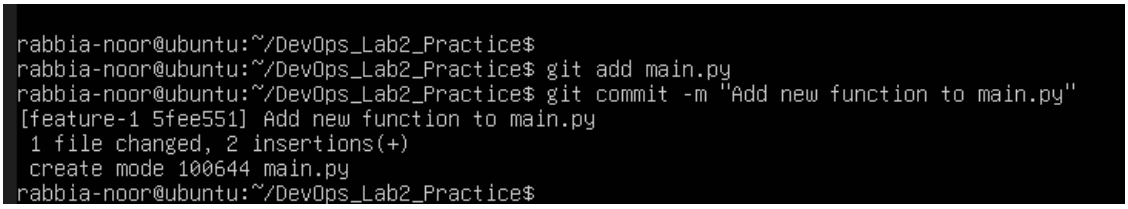


Figure 19: *A new function was added to main.py, then staged with git add and committed on the feature-1 branch. The commit output shows the changes were saved only on this branch, and main remains unchanged.*

Step 3: Switch back to main and merge feature-1.

```
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git merge feature-1
Updating e8c796b..5fee551
Fast-forward
 main.py | 2 ++
 1 file changed, 2 insertions(+)
 create mode 100644 main.py
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 20: *After switching back to main with git checkout, the feature-1 branch was merged using git merge. Git reports a fast-forward merge and shows main.py was updated, so main now contains the new function.*

Step 4: Push all branches to the remote repository.

```
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 368 bytes | 30.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:Rabbia-Noor/DevOps_Lab2_Practice.git
   e8c796b..5fee551  main -> main
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git push origin feature-1
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature-1' on GitHub by visiting:
remote:      https://github.com/Rabbia-Noor/DevOps_Lab2_Practice/pull/new/feature-1
remote:
To github.com:Rabbia-Noor/DevOps_Lab2_Practice.git
 * [new branch]      feature-1 -> feature-1
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 21: *Both branches were pushed to the remote repository using git push origin main and git push origin feature-1. This uploads the merged work to GitHub so it is backed up and visible to the rest of the team.*

Task 10: Pull Request and Branch Review (GitHub GUI)

Objective: Use a Pull Request to review and merge a branch using the GitHub interface.

Step 1: On GitHub, create a Pull Request from feature/db-connection to main.

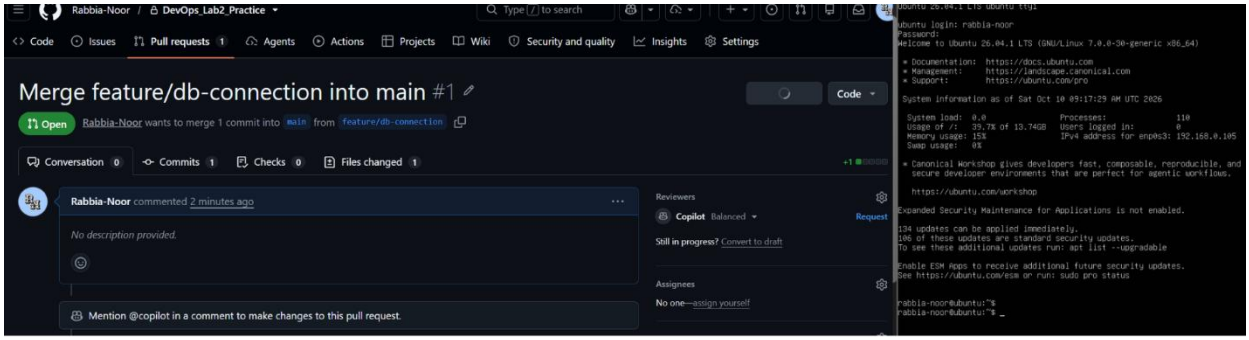


Figure 22: *A Pull Request was opened on GitHub to merge the branch feature/db-connection into main. The Pull Request page lists the commit and changed files, so teammates can review the changes before they are merged.*

Step 2: Review the Pull Request and merge it using the GitHub GUI.

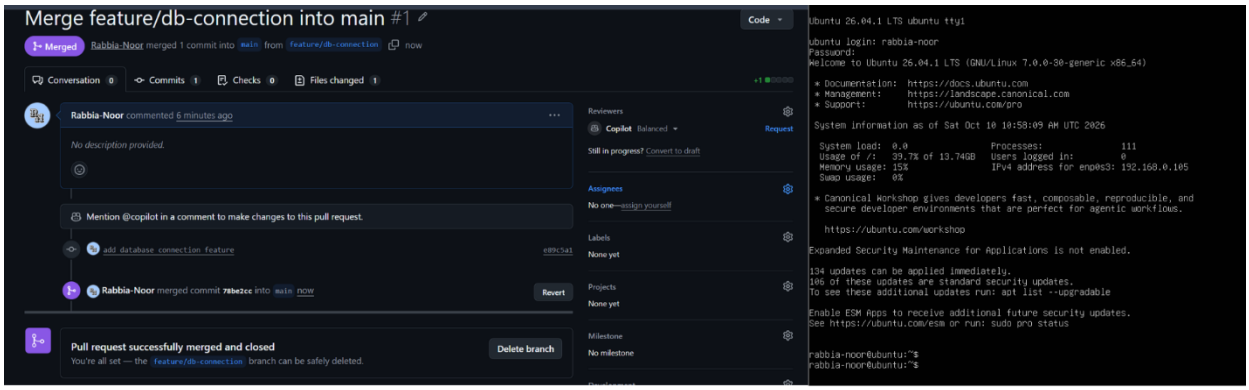


Figure 23: *The Pull Request was reviewed and merged using the GitHub interface. The purple Merged label and the success message confirm that the changes from feature/db-connection are now part of the main branch.*

Step 3: After merging, delete the feature/db-connection branch using the GitHub GUI.

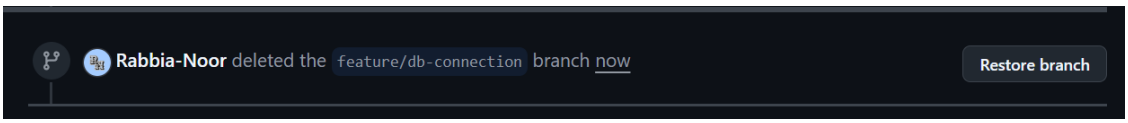


Figure 24: *After the merge, the feature/db-connection branch was deleted using the GitHub interface. The page shows the deletion event with a Restore branch option. Deleting merged branches keeps the repository clean.*

Task 11: Detailed Branch Strategy (Develop/Staging)

Objective: Simulate a professional branching strategy with develop, staging and production (main) branches.

Step 1: Create the branches develop, staging, feature/* and bugfix/* and show the branch list.

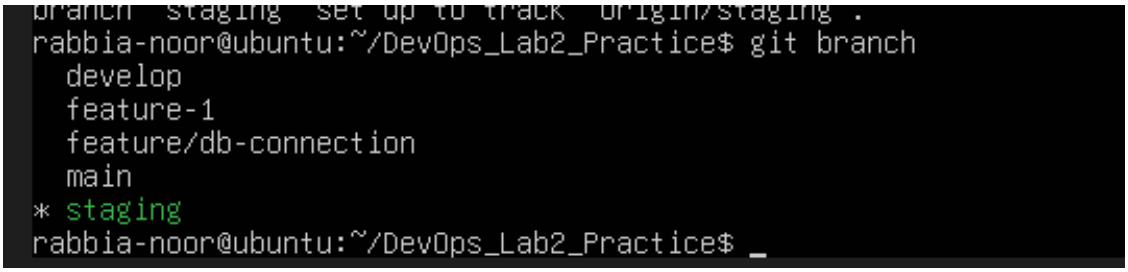


Figure 25: *The branch list shows the structure of the branching strategy: main for production, develop for integration, staging for testing, and separate feature and bugfix branches. Each branch has a clear purpose in the workflow.*

Step 2: Merge the feature work into develop, then merge develop into staging.

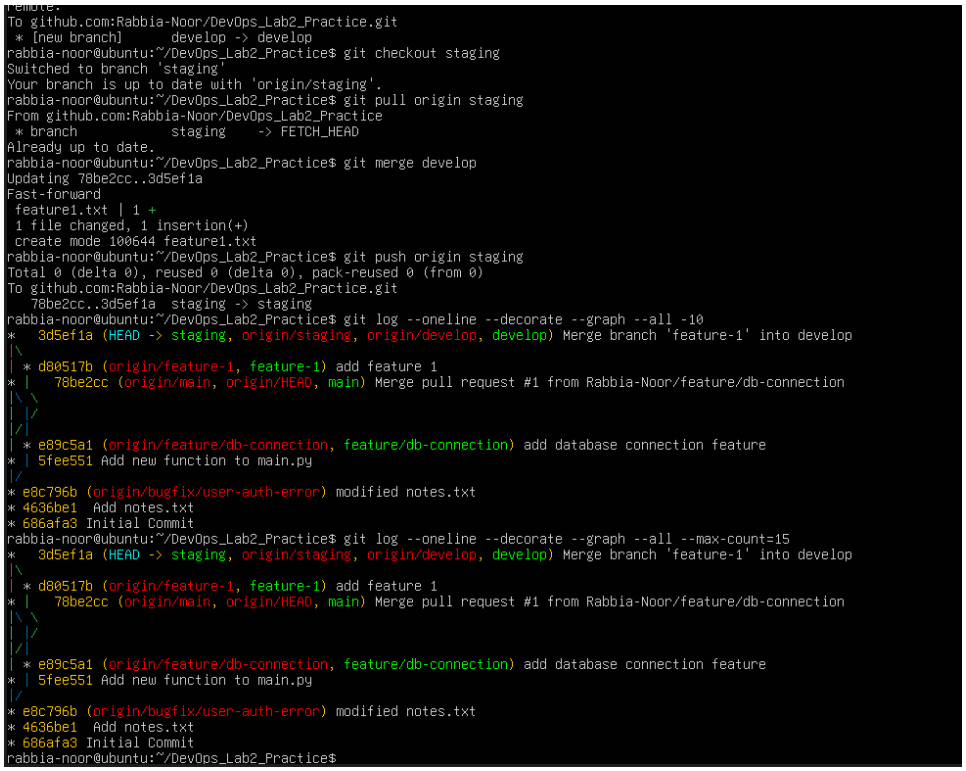


Figure 26: *The feature branch was merged into develop, and develop was then merged into staging. The commands and the commit history graph show the changes moving step by step through the workflow for testing.*

Step 3: Merge staging into main (production) and push.

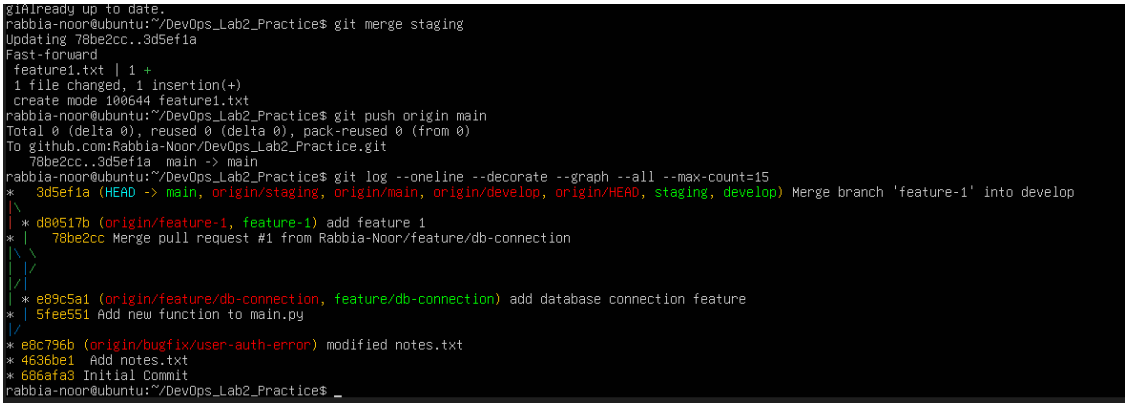


Figure 27: *Finally, staging was merged into main and pushed to GitHub, which represents releasing to production. The commit graph shows all earlier merges, confirming the full flow from feature to develop, staging and main.*

Task 12: Code Review Workflow

Objective: Practice the complete code review cycle using Pull Requests.

Step 1: Create a Pull Request from a feature branch into main with a clear title and description.

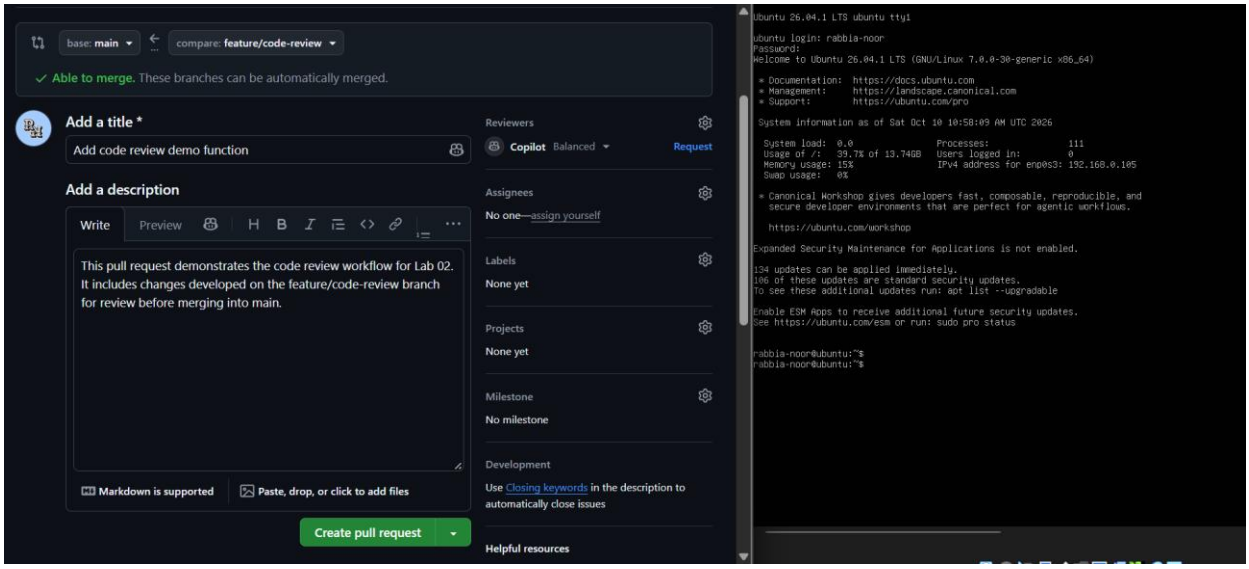


Figure 28: *A Pull Request was created from the feature/code-review branch into main with a clear title and a bullet-point description. A good description helps reviewers quickly understand what changed and why.*

Step 2: Assign a reviewer (teammate or second account).

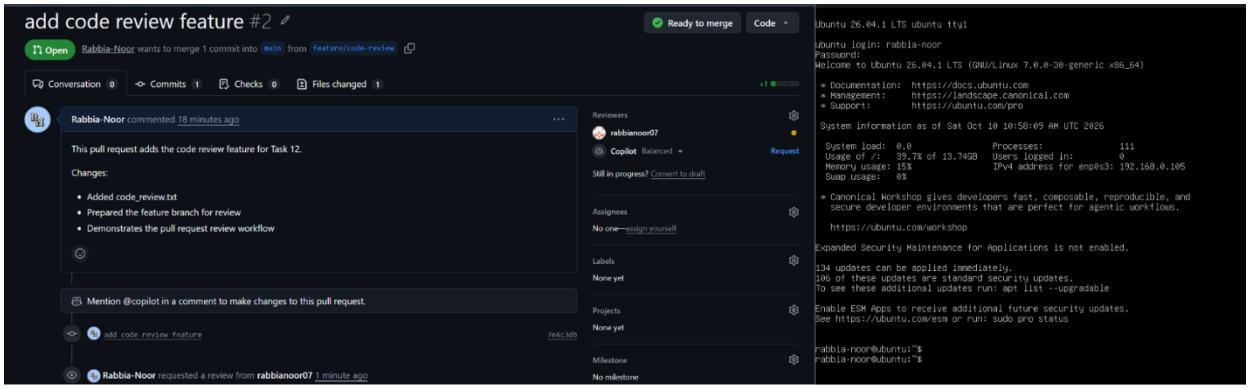


Figure 29: *A reviewer was assigned to the Pull Request using the Reviewers section on the right. The assigned reviewer is notified and is responsible for checking the code before it can be merged into main.*

Step 3: Reviewer action: request changes and add comments.

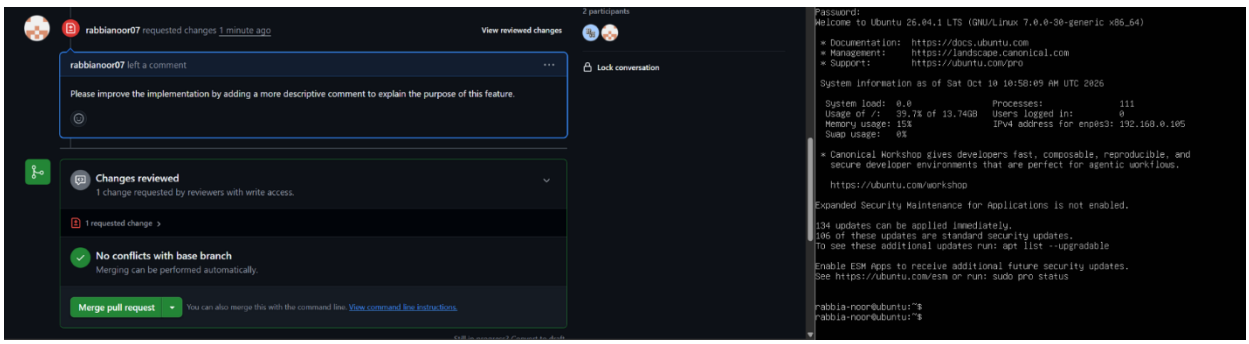


Figure 30: *The reviewer chose Request changes and left a comment asking for a more descriptive explanation of the feature. The Pull Request is marked as needing changes, so the author must fix the issues first.*

Step 4: Reviewer action: reject or close a separate practice Pull Request.

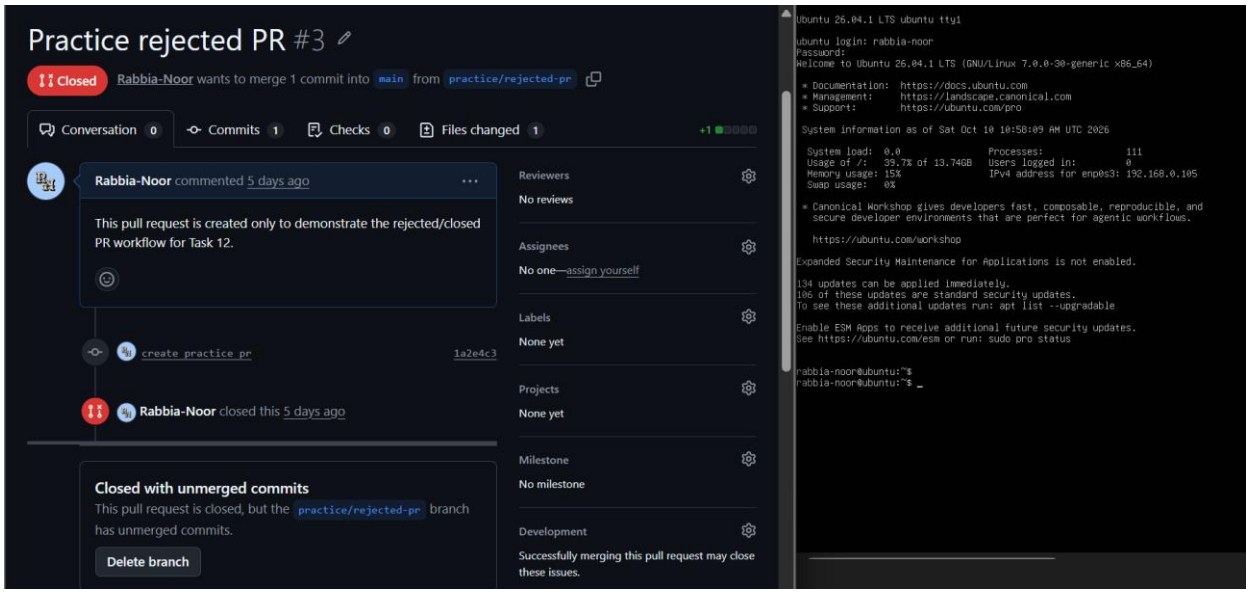


Figure 31: *A separate practice Pull Request was closed without merging to demonstrate the rejected state. The timeline shows it was closed. This is how a change that should not be accepted is turned down safely.*

Step 5: Author action: fix the issues in the same branch and push new commits. The Pull Request updates automatically.

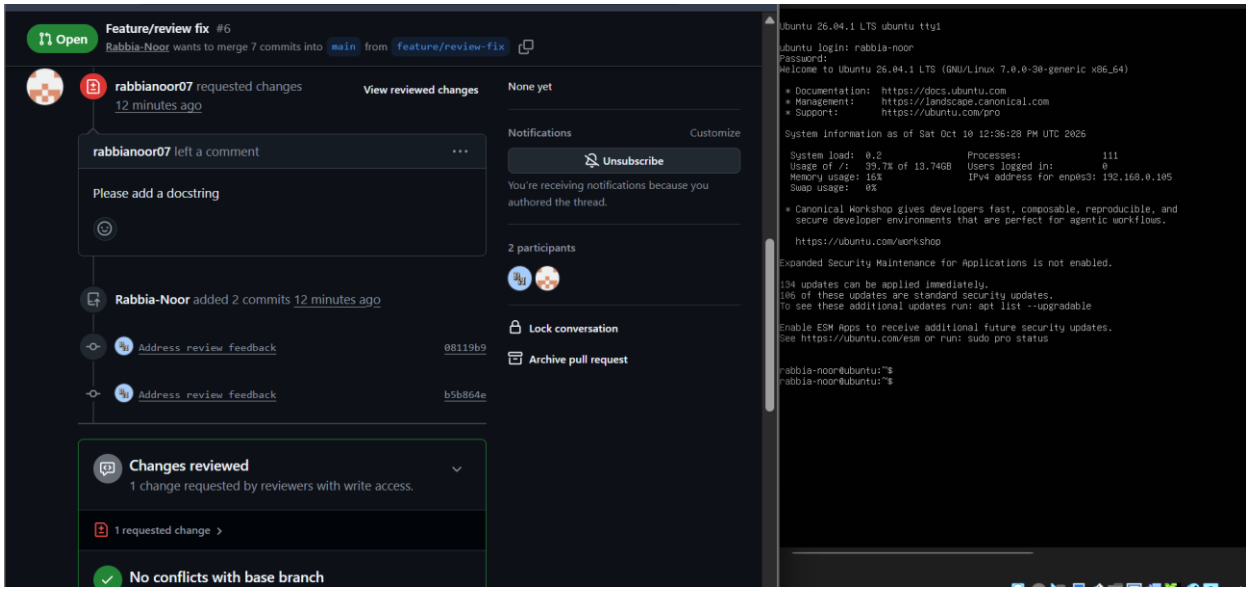


Figure 32: *After the reviewer asked for changes, new commits were pushed to the same branch. The Pull Request updated automatically to include them, so the reviewer can check the fixes without creating a new Pull Request.*

Step 6: The reviewer approves the changes and the Pull Request is merged using the GitHub UI (merge commit).

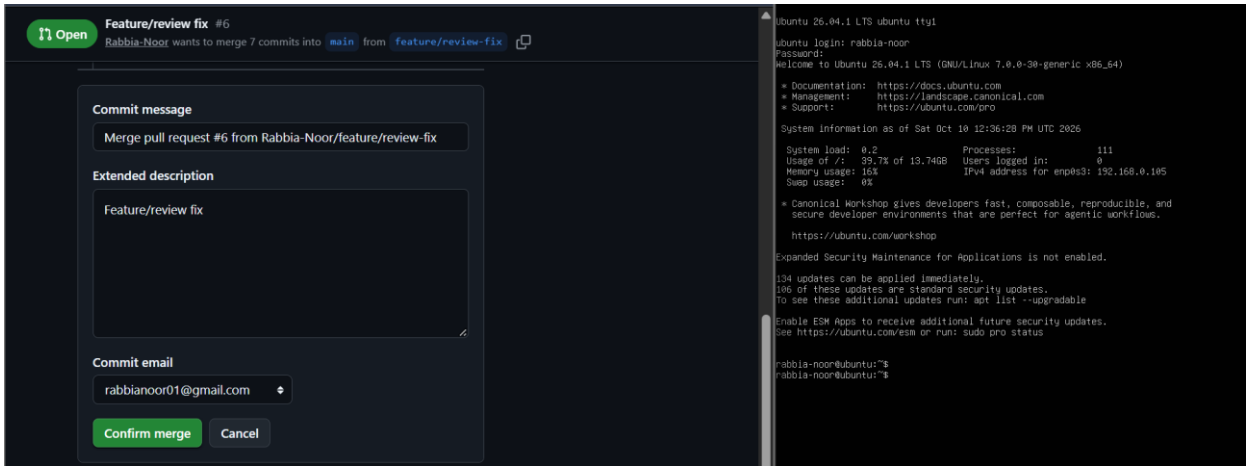


Figure 33: *The reviewer approved the changes and the Pull Request was merged into main using the GitHub interface. The Merged message and the new merge commit confirm that the reviewed code is now part of main.*

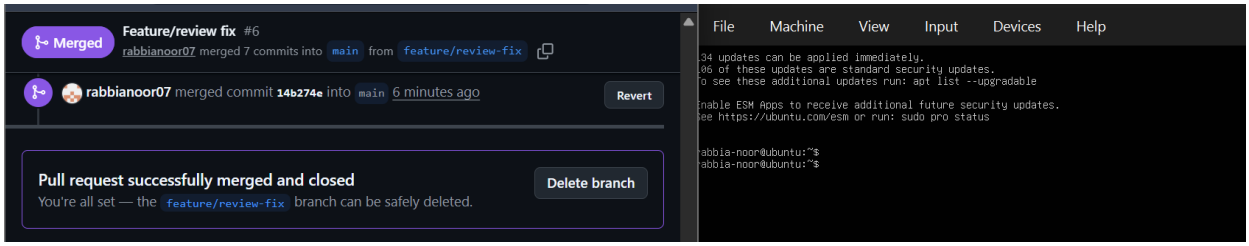


Figure 34: *The Pull Request list shows the request "add code review feature" as merged and approved. This gives a quick record that the review process was completed and the change was accepted into the project.*

Step 7: Delete the merged Pull Request branch.

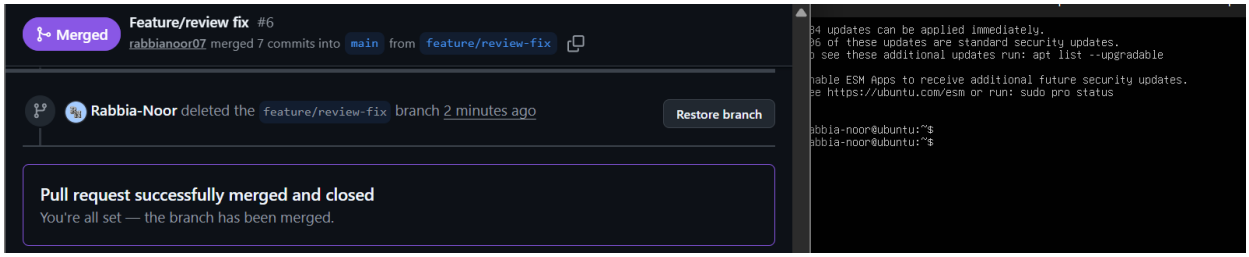


Figure 35: *After the Pull Request was merged, the feature/code-review branch was deleted from GitHub. The timeline shows the branch deletion event. Removing finished branches avoids clutter and confusion in the repository.*

Task 13: Branch Cleanup Best Practices

Objective: Remove merged branches to keep the repository clean.

Step 1: Delete a merged branch through the GitHub UI.



Figure 36: *A merged branch was deleted through the GitHub UI, and the updated branch list no longer includes it. Only the active branches remain. This is a best practice to keep the remote repository tidy after merging.*

Step 2: Delete another completed remote branch from the terminal.

```
Your branch is up to date with 'origin/main'.
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git push origin --delete bugfix/user-auth-error
To github.com:Rabbia-Noor/DevOps_Lab2_Practice.git
 - [deleted]         bugfix/user-auth-error
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git checkout main
```

Figure 37: *A remote branch was deleted from the terminal using git push origin --delete. The output marks the branch as [deleted]. This does the same job as the GitHub button, but directly from the command line.*

Exam Evaluation Questions

Advanced Branching & Merge Verification

- Create a new branch in your repository

```
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git checkout main
Already on 'main'
Your branch is up to date with 'origin/main'.
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git pull origin main
From github.com:Rabbia-Noor/DevOps_Lab2_Practice
 * branch            main       -> FETCH_HEAD
Already up to date.
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git checkout -b Q1-branch
Switched to a new branch 'Q1-branch'
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git branch
* Q1-branch
  develop
  feature/code-review
  feature/db-connection
  main
  practice/rejected-pr
  staging
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$
```

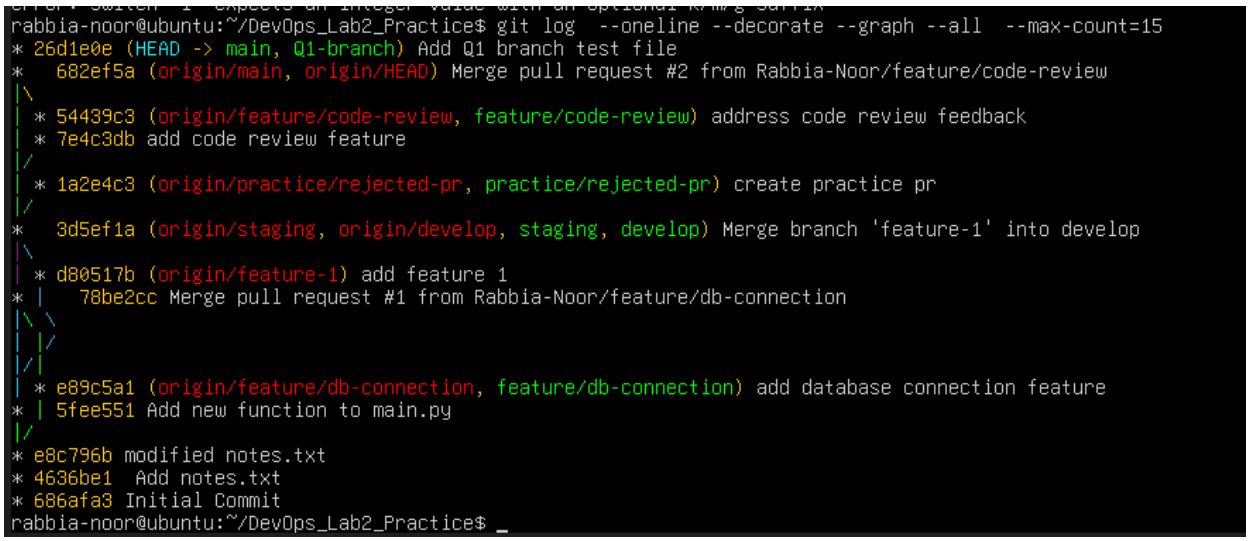
Make a small change in a file and commit it

```
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ echo "Q1 branching and merge verification" > Q1_test.txt
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ cat Q1_test.txt
Q1 branching and merge verification
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git add Q1_test.txt
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git commit -m "Add Q1 branch test file"
[Q1-branch 26d1e0e] Add Q1 branch test file
 1 file changed, 1 insertion(+)
 create mode 100644 Q1_test.txt
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git log --oneline -2
26d1e0e (HEAD -> Q1-branch) Add Q1 branch test file
682ef5a (origin/main, origin/HEAD, main) Merge pull request #2 from Rabbia-Noor/feature/code-review
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$
```

Merge this branch back into the main branch.

```
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git merge Q1-branch
Updating 682ef5a..26d1e0e
Fast-forward
 Q1_test.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 Q1_test.txt
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git log  --online --decorate --graph  -5
fatal: unrecognized argument: --online
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git log --oneline --decorate --graph -5
* 26d1e0e (HEAD -> main, Q1-branch) Add Q1 branch test file
*   682ef5a (origin/main, origin/HEAD) Merge pull request #2 from Rabbia-Noor/feature/code-review
|\
| * 54439c3 (origin/feature/code-review, feature/code-review) address code review feedback
| * 7e4c3db add code review feature
|/
*   3d5ef1a (origin/staging, origin/develop, staging, develop) Merge branch 'feature-1' into develop
|\
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$
```

Show the history of commits in a way that verifies the merge.


rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ git log --oneline --decorate --graph --all --max-count=15
* 26d1e0e (HEAD -> main, Q1-branch) Add Q1 branch test file
*   682ef5a (origin/main, origin/HEAD) Merge pull request #2 from Rabbia-Noor/feature/code-review
|\
| * 54439c3 (origin/feature/code-review, feature/code-review) address code review feedback
| * 7e4c3db add code review feature
|/
| * 1a2e4c3 (origin/practice/rejected-pr, practice/rejected-pr) create practice pr
|/
*   3d5ef1a (origin/staging, origin/develop, staging, develop) Merge branch 'feature-1' into develop
|\
| * d80517b (origin/feature-1) add feature 1
* | 78be2cc Merge pull request #1 from Rabbia-Noor/feature/db-connection
|\ \
| |/
|/|
| * e89c5a1 (origin/feature/db-connection, feature/db-connection) add database connection feature
* | 5fee551 Add new function to main.py
|/
* e8c796b modified notes.txt
* 4636be1 Add notes.txt
* 686afa3 Initial Commit
rabbia-noor@ubuntu:~/DevOps_Lab2_Practice$ _